

嵌入式计算机系统及实验

基于 MSP430 的心电检测系统

目录

1. 实验目的	2
2. 实验内容	2
2.1 系统架构设计	2
2.2 硬件以及接口	2
2.2.1 ADC 接线	2
2.2.2 TFT 接口	3
2.3 点阵汉字生成	3
3. 流程图	6
4. 主要程序解读	6
4.1 初始化	6
4.2 定时器中断	9
4.3 计算心率	11
5. 结果展示	13
5.1 电极片位置	13
5.2 心电图展示	14
5.3 UART 串口通信结果及后处理	15
6. 困难与解决方案	17
6.1 问题一：代码可读性与小组合作	17
6.2 问题二：心率计算	17
6.3 问题三：心率平均计算	17
6.4 问题四：串口通信过程中的数据转化问题	17

基于 MSP430 的心电检测系统

1. 实验目的

1. 利用 AD8232 心电图监测芯片监测心电信号
2. 利用 MSP430 内部 ADC 模块采集 AD8232 输出的心电放大信号
3. 在 MSP430 的 TFT 点阵屏上显示心电波形，并计算心率
4. 通过 MSP430 UART-RS232-USB 接口将心电信号传输到电脑端，并利用 Python 进行分析和可视化

2. 实验内容

2.1 系统架构设计

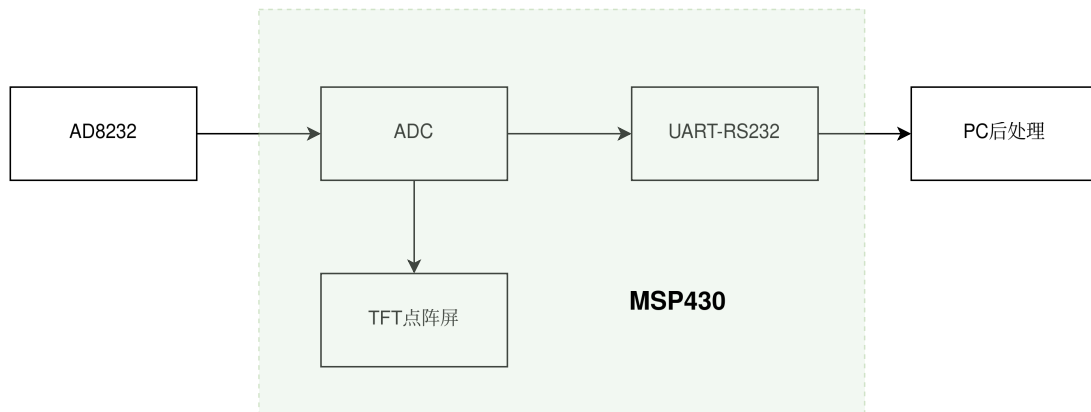


图 1 系统架构图

采集流程和架构：

1. AD8232 芯片采集心电信号，输出模拟信号
2. 将 AD8232 输出的模拟信号作为 MSP430 内部 ADC 模块的输入
3. ADC 模块将模拟信号转换为数字信号
4. UART 输出数字信号到 PC 端
5. 对 MSP430 的输出做处理，在 TFT 点阵屏上显示心电波形

2.2 硬件以及接口

2.2.1 ADC 接线

AD8232 芯片输出接 ADC 通道 12: P7.4。心电芯片的 3.3V 电源接到 MSP430VCC，GND 接到 MSP430 的 GND

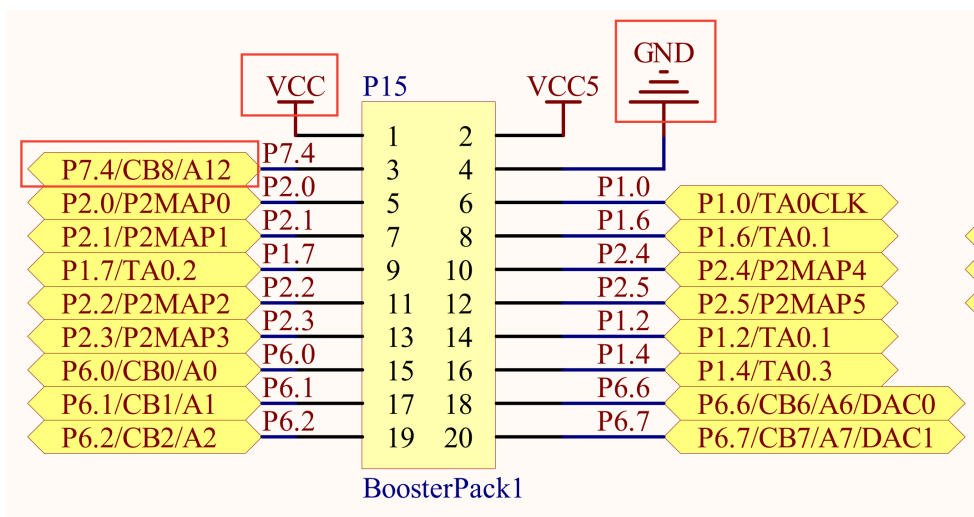


图2 MSP430 ADC 接口

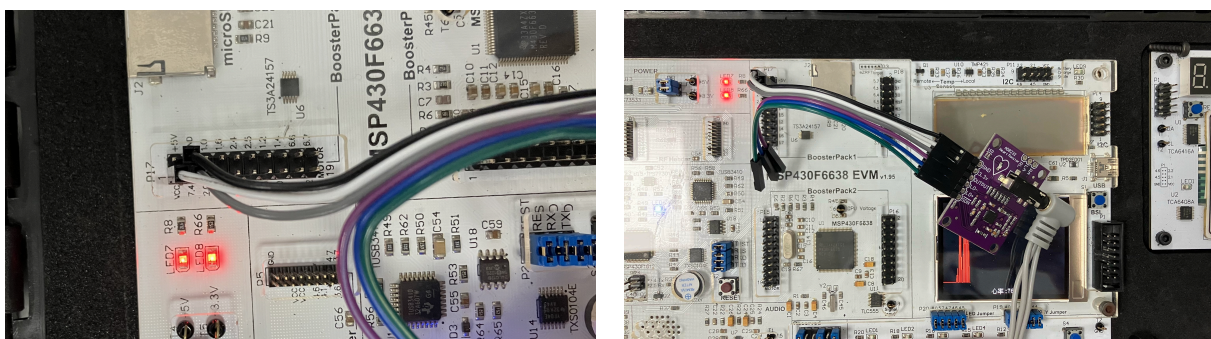


图3 ADC 实物接口

2.2.2 TFT 接口

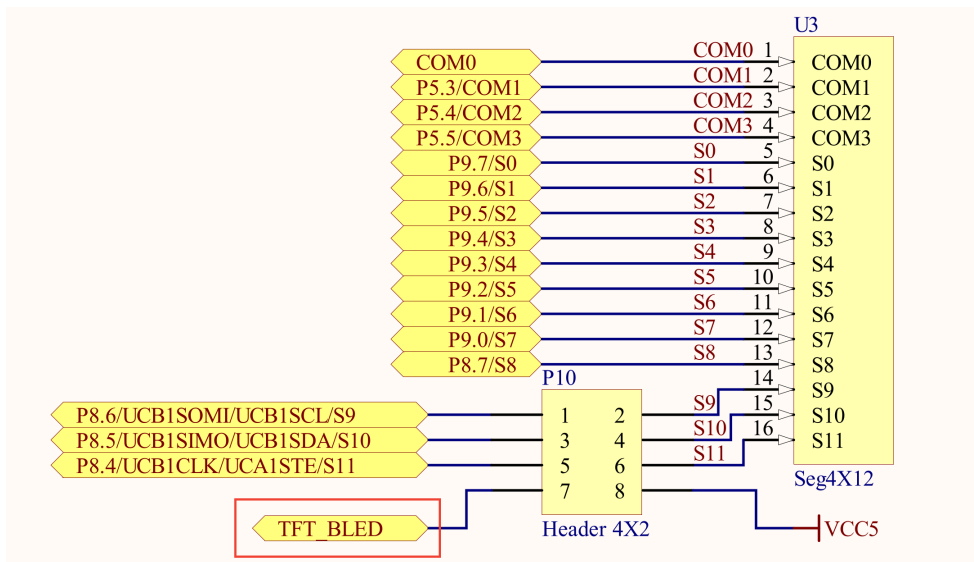


图4 MSP430 TFT 输出接口

2.3 点阵汉字生成

为了在 TFT 屏上显示汉字“心率”，需要将汉字转换为点阵图像，使用点阵图像的方式进行显示

在网上下载 hzk16 点阵汉字库，利用 Python 脚本将其转换为 C 语言数组，存储在头文件 `ecg_words.h` 中

处理代码如下：

```
1  import numpy as np
2  # 定义点阵汉字获取函数，从hzk16s字库文件中获取指定汉字的点阵矩阵
3  def get_hzk_matrix(char, hzk_data_path='hzk16s', size=16):
4      if size not in [16, 32]:
5          raise ValueError("size must be either 16 or 32")
6      length = size // 8
7
8      # 获取汉字的GB2312编码
9      gb2312 = char.encode('gb2312')
10     zone_code, bit_code = gb2312[0], gb2312[1]
11     offset = ((zone_code - 0xA1) * 94 + (bit_code - 0xA1)) * (size * length)
12
13     # 读取字库文件
14     with open(hzk_data_path, 'rb') as f:
15         f.seek(offset)
16         font_data = f.read(length * size)
17
18     # 转换为点阵矩阵
19     matrix = []
20     for i in range(size):
21         row_data = font_data[i*length:i*length+length]
22         row = []
23         for byte in row_data:
24             for j in range(8):
25                 row.append(byte >> (7-j) & 1)
26         matrix.append(row[:size]) # 只取前16或32位
27
28     # 打印点阵矩阵，查看效果
29     for row in matrix:
30         print(' '.join(['█' if pixel else '_' for pixel in row]))
31
32     return np.array(matrix)
33
34 words = ["心", "率"]
35 # 写入C语言头文件，便于嵌入到MSP430程序中
36 with open(r"ecg_words.h", "w") as f:
37     f.write(f"unsigned char const words" + "[] = {\n")
38     for i, word in enumerate(words):
39         matrix = get_hzk_matrix(word)
40         for row in matrix:
41             for pixel in row:
42                 # 打印颜色，黑底白字
43                 if pixel:
```

 Python

```

44         for c in color:
45             f.write(f"0xff, ")
46     else:
47         for _ in range(3):
48             f.write("0x00, ")
49         f.write("\n")
50     f.write("\n};\n")

```

最终效果如下:

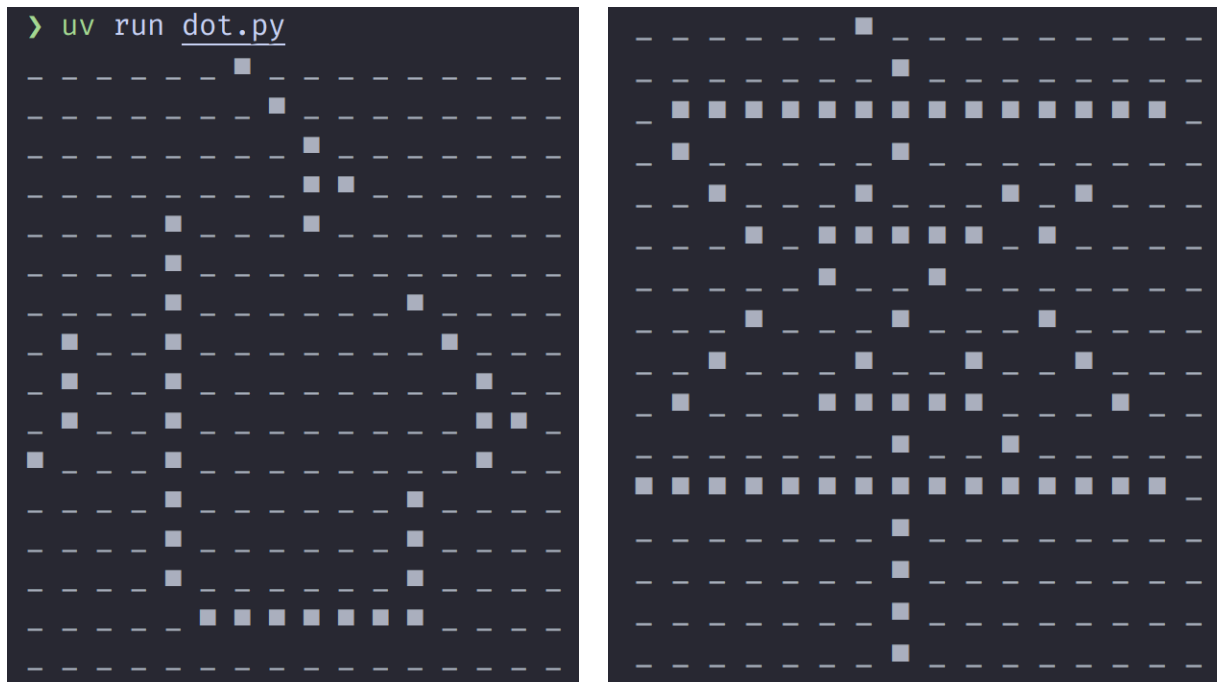


图5 点阵汉字示例

3. 流程图

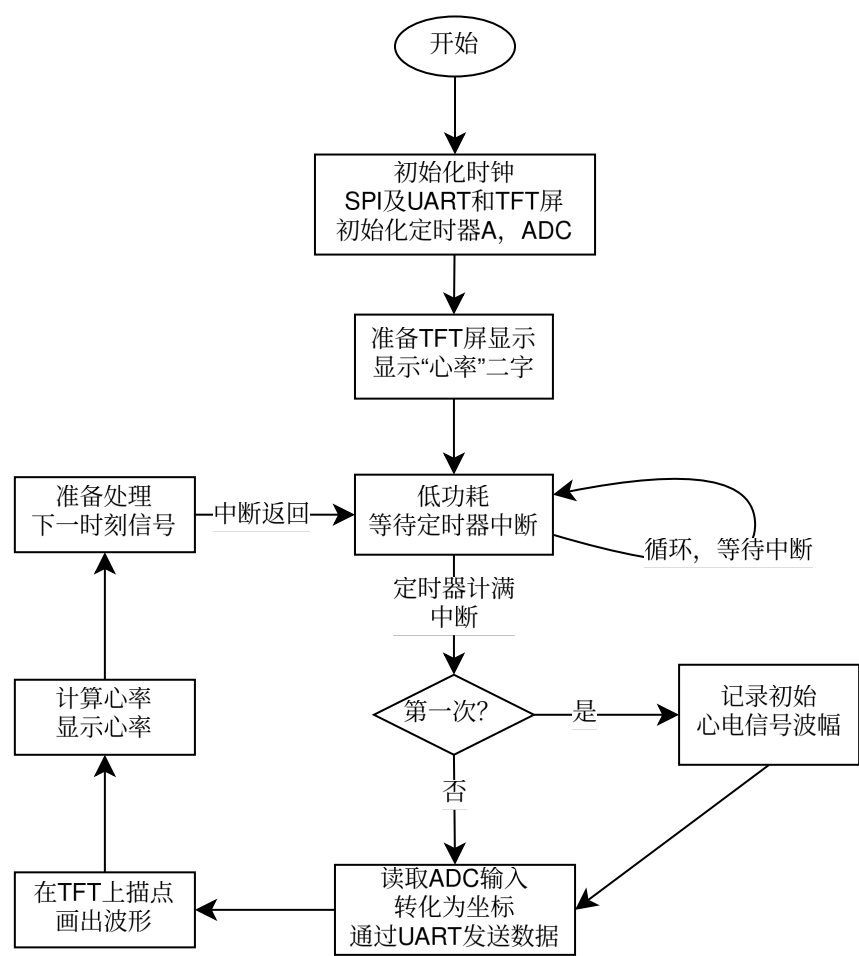


图 6 系统软件流程图

4. 主要程序解读

整体采用模块化设计理念，将所有复杂函数进行封装，方便阅读和扩展

4.1 初始化

首先对所有使用到的硬件资源进行初始化，包括

- 内部时钟
- SPI, UART 串口
- TFT 屏
- TimerA
- ADC

在初始化过程前需要关闭中断，在初始化结束后再打开中断；这是为了防止中断输入对初始化造成不必要的干扰

在初始化结束后准备 TFT 输出，预先显示汉字提示；最后进入 LPM0，待机准备接收中断信号
主程序如下

1 // 使用到的函数头

CC

```

2 void UART_RS232_Init(void);           // 串口初始化
3 void initAll();                       // 所有的初始化
4 void show_words(uint16_t sx, uint16_t sy, uint16_t size); // 显示汉字
5 void initClock();                     // 时钟初始化
6 void display_heart_rate();            // 显示心率到TFT
7 void draw_wave();                     // 在TFT上画波形
8 int calc_heart_rate();                // 计算心率
9
10 void main(void)
11 {
12     WDTCTL = WDTPW + WDTHOLD;          // 关闭看门狗，防止自动复位
13     _DINT();                           // 禁用总中断，防止初始化被打断
14     initAll();                         // 所有的初始化操作
15     _EINT();                           // 初始化完成，使能总中断
16     __bis_SR_register(LPM0_bits + GIE); // 进入低功耗模式LPM0并开启中断
17     // 等待中断唤醒
18 }

```

其中 `initAll()` 是一个封装好的总体初始化函数，实现如下

```

1 // 所有的初始函数
2 void initAll()
3 {
4     UART_RS232_Init();                 // 初始化串口
5
6     // 初始化TimerA
7     TA0CTL |= MC_1 + TASSEL_2 + TACLK; // 时钟为SMCLK，比较模式，开始时清零计数器
8     TA0CCTL0 = CCIE;                  // 比较器中断使能
9     TA0CCR0 = 50000;                   // 比较值设为50000，相当于50ms的时间间隔
10                                         // 但在initClock之后由于系统时钟改变
11                                         // 频率变为20倍，这相当于2.5ms的时间间隔
12     // 初始化ADC
13     ADC12CTL0 |= ADC12MSC;              // 自动循环采样转换
14     ADC12CTL0 |= ADC12ON;               // 启动ADC12模块
15     ADC12CTL1 |= ADC12CONSEQ1;          // 选择单通道循环采样转换
16     ADC12CTL1 |= ADC12SHP;              // 采样保持模式
17     ADC12MCTL0 |= ADC12INCH_12;         // 选择通道12
18     ADC12CTL0 |= ADC12ENC;              // 开始转换
19
20     initClock();                       // 初始化系统时钟（使用XT1和XT2作为参考）
21     initTFT();                         // 初始化TFT显示屏
22
23     etft_AreaSet(0, 0, 319, 239, 0);    // 清屏（设置整个显示区域颜色）
24     show_words(127, 223, 16);           // 显示“心率”两个字
25
26 }

```

其中 `UART_RS232_Init()`, `initClock()`, `initTFT()` 的实现和平时实验中并无区别, 不赘述
特别需要注意的是, 因为在初始化时钟中有

```
1 UCSCTL2 = 20000000 / (4000000 / 16);
```

这表明期望得到 DCO 的输出频率为 20MHz, 而在 MSP430 中, `SMCLK` 默认由 DCO 提供, 这使得定时器的频率也变为 20MHz (比默认高 20 倍), 因此

```
1 TA0CCR0 = 50000;
```

不再是定 50ms, 而是 $\frac{50}{20} = 2.5\text{ms}$, 这在后文会再次提到, 这里先做引入

显示汉字的函数 `show_word()` 实现如下, 其中 `words` 是汉字的点阵数组, 在头文件中定义

```
1 // 显示汉字的函数
2 void show_words(uint16_t sx, uint16_t sy, uint16_t size)
3 {
4     uint16_t cx, cy;
5     uint16_t words_len = 2;
6
7     int i;
8     for (i = 0; i < words_len; i++)
9     {
10         cx = 0;
11         cy = 0;
12         // 屏幕是横的, XY要对调
13         tft_SendCmd(TFTREG_WIN_MINX, sx);           // x start point
14         tft_SendCmd(TFTREG_WIN_MINY, sy);           // y start point
15         tft_SendCmd(TFTREG_WIN_MAXX, sx + size - 1); // x end point
16         tft_SendCmd(TFTREG_WIN_MAXY, sy + size - 1); // y end point
17         tft_SendCmd(TFTREG_RAM_XADDR, sx);           // x start point
18         tft_SendCmd(TFTREG_RAM_YADDR, sy);           // y start point
19         tft_SendIndex(TFTREG_RAM_ACCESS);
20
21         uint16_t color;
22         while (1)
23         {
24             if (cx ≥ size) {
25                 cx = 0;
26                 cy++;
27                 if (cy ≥ size) // 一个字符发送完毕
28                 {
29                     sx += size;
30                     if (sx ≥ TFT_YSIZE) // 越过行末
31                     {
32                         sx = 0;
33                         sy += size;
34                     }
35                     break;

```

```

36         }
37     }
38     uint16_t pos = i * size * size * 3 + cy * size * 3 + cx * 3;
39     color = etft_Color(words[pos], words[pos + 1], words[pos + 2]);
40     tft_SendData(color);
41     cx++;
42 }
43 }
44 }

```

4.2 定时器中断

采用定时器中断采样法，当定时器 A 计满时进入中断，采样 ADC 的输入并进行后续的流程本模块按顺序实现了

- 读取 ADC 输入
- 转化原始输入为屏幕上的坐标
- 对 UART 的发送 BUFFER 进行赋值，发送原始数据到 PC、
- 在 TFT 屏上画出曲线
- 计算心率并显示在 TFT 屏上

由于我们的画图函数需要前一时刻的信号和当前时刻的信号，故第一次采样时需要额外进行一次采样

这是本系统的核心部分，中断例程如下

```

1  // 定义 Timer_A中断服务函数，每次计满都会画曲线
2  #pragma vector = TIMER0_A0_VECTOR
3  __interrupt void Timer_A (void)
4  {
5      // 第一次进入，需要额外计算一个点
6      if (is_first_time == 1)          // 如果是第一次进入
7      {
8          ADC12CTL0 |= ADC12SC;          // 读取下一次数据
9          value = ADC12MEM0;              // 赋值，注意这里value最大值是2^12=4096
10                                     // 因为是12bit的ADC输入
11          is_first_time = 0;              // 清除，后续不再需要
12          prev_wave = 207 - value / 20;    // 207是设定的能在屏上显示的最下方
13                                     // 20来自于4096/207=20
14                                     // 可以将ADC读到的数据放缩到屏幕的范围内
15                                     // 207我们设定的显示的最下方，为了美观而设计
16
17          // 除20以后强制转换为8位数据，如果不转换，UCA1TXBUF
18          // 会把高4位截断，最后绘图无法绘制出正确的波形
19          UCA1TXBUF = (uint8_t) (value / 20);          // 赋值给发送缓冲，准备发送
20      }
21
22      // 处理当前时刻的幅值
23      ADC12CTL0 |= ADC12SC;              // 启动 ADC转换

```

```

24  value = ADC12MEM0;           // 将ADC转换结果赋值给变量value
25  curr_wave = 207 - value / 20; // 和上面同理
26  // 这里转换同理，需要转成8位数据以便串口发送
27  UCA1TXBUF = (uint8_t)(value / 20); // 串口发送
28
29  // 画波形，显示心率
30  draw_wave();                 // 画波形
31  display_heart_rate();        // 显示心率
32
33  // 准备下一次处理
34  t++;                         // t坐标加 1，准备处理下一个坐标
35  prev_wave = curr_wave;       // 时刻更新，幅值也要更新
36  }

```

画波形的函数 `draw_wave()` 实现如下，这一部分将每次计算出的点坐标描在 TFT 屏上并连成线

```

1  // 画波形使用的存储区
2  volatile unsigned int value = 0; // 读取的ADC的输入
3  volatile unsigned int is_first_time = 0; // 是否是第一次采集信号，因为画波形需要两个
    时刻的幅值
4  volatile unsigned int prev_wave = 0; // 前一时刻的幅值
5  volatile unsigned int curr_wave = 0; // 当前时刻的幅值
6  volatile unsigned int t = 0; // 时间变量
7  volatile unsigned int median = 0; // 两个时刻的幅值中间值，用于平滑曲线
8
9  // 画波形的函数
10 void draw_wave()
11 {
12     if (t ≥ 319) // 如果已经画到了最右边
13     {
14         t = 0; // 将t设为0，从左端开始重新画
15         etft_AreaSet(0, 0, 319, 223, 0); // 更新当前的图像窗口
16     }
17     median = (prev_wave + curr_wave) / 2; // 将median设为prev_wave和curr_wave的平均
    值
18
19     if (prev_wave < curr_wave) // 如果prev_wave小于 curr_wave
20     {
21         etft_AreaSet(t, prev_wave, t, median, 63488); // 画前一时刻的波形，设置为红色
22         etft_AreaSet(t+1, median, t+1, curr_wave, 63488); // 画当前时刻的波形，设置为红
    色
23     }
24     else // 如果大小相反需要反过来
25     {
26         etft_AreaSet(t, median, t, prev_wave, 63488);
27         etft_AreaSet(t+1, curr_wave, t+1, median, 63488);
28     }

```

为了让曲线更加精确，引入了 `median` 作为两个时刻幅值的平滑过渡，原理大致如下

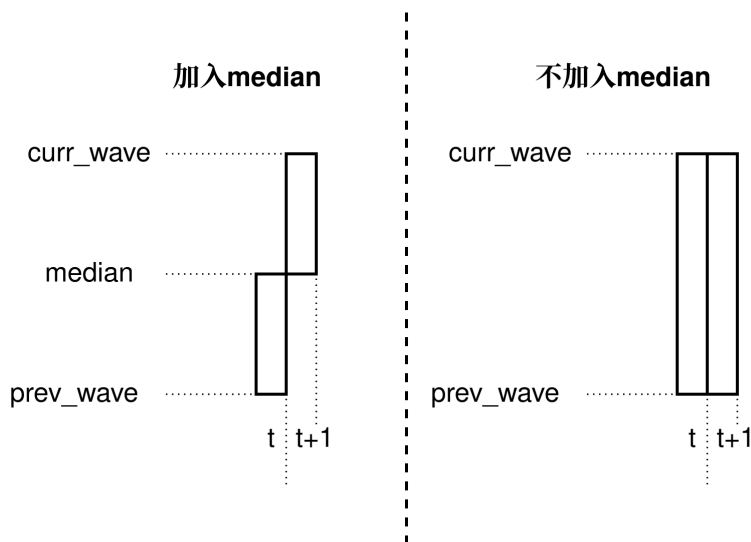


图7 TFT点阵绘制波形原理

显示心率的函数 `display_heart_rate()` 实现如下，这部分将 `calc_heart_rate()` 函数计算得到的心率显示出来，具体的处理比较常规

```
1  char dispaly_rate[5]=":000";           // 存储心率的每一位
2
3  void display_heart_rate()
4  {
5      int heart_rate = calc_heart_rate(); // 计算curr_wave和prev_wave的脉冲计数
6      if (heart_rate > 0)                 // 仅当有心率时才计算
7      {
8          dispaly_rate[1] = heart_rate / 100 + '0'; // 取心率的百位
9                                                    // 加上'0'的变为数字字符而不是整型
10         dispaly_rate[2] = (heart_rate % 100) / 10 + '0'; // 取十位
11         dispaly_rate[3] = heart_rate % 10 + '0'; // 取个位
12     }
13     etft_DisplayString(dispaly_rate, 161, 223, 65535, 0); // 显示心率
14 }
```

虽然定时器的中断例程需要尽量简短，以防止中断无法即时响应，但是本系统中与 TFT 屏通讯的时间开销非常大，大于定时器定的 2.5ms；在实际测试中我们发现描完 320 个点（TFT 的宽度）大约需要 3.1s，也就是说大约 10ms 会进入一次中断，每次中断例程大约持续 7.5ms

我们尝试过在主程序中进行所有的耗时操作，但是这会导致中断对通讯造成干扰；如果在主程序的这些操作过程中屏蔽中断，效果和原理和在中断例程中进行操作是一样的（因为进入中断例程后会屏蔽其他中断信号），因此为了程序阅读的便利，我们最后选择原方案

4.3 计算心率

接收 ADC 的输入后需要进一步进行心率的计算，我们采用阈值检测法进行心率计算

在实验中我们发现心电波形存在很明显的特点

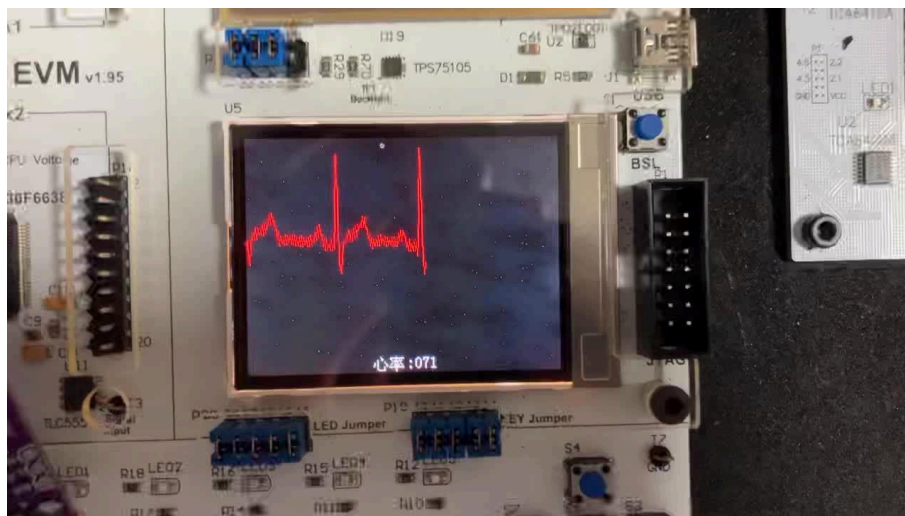


图 8 心电波形示例

信号的高峰（R 波）远高于其他成分，可以用峰峰间隔（RR）作为心跳周期测量的依据

我们采用了简单而有效的三点峰值检测和时域非最大值抑制进行 R 波的检测：依次采样三个点，若中间时刻的点是最高且高于阈值，那么就是高峰；相邻的两个 R 波出现时间差存在限制，防止噪声和其他波峰的干扰

由于正常人心跳存在心跳变异率（HRV），相邻两次的心跳不是等间隔的，会存在一定方差，因此仅检测一个 RR 间隔并不能说明实际心率

为了解决这个问题，我们采用了时域均值相关法进行近似估计：采集一个窗口内的每一次计算得到的心率，取平均得到这个时间段内的代表心率

函数 `calc_heart_rate()` 实现如下，为了更精确可以将 `peak_time` 设得更长，但会损失时效性

```
1 // 计算心率的存储区
2 volatile int prev2 = 0; // 前两个时刻的幅值
3 volatile int prev1 = 0; // 前一个时刻的幅值
4 volatile int peak_times[5] = {0, 0, 0, 0, 0}; // 峰值时间
5 // 记录一段时间内的每个峰峰时间，更加稳定
6 volatile int pointer = 0; // 峰值时间的指针
7 volatile unsigned int curr_time = 0; // 当前采样点时间
8 volatile int threshold = 30; // 检测峰值的阈值
9
10 // 计算心率的函数
11 int calc_heart_rate()
12 {
13     // 简单三点峰值检测：中点高于两边并大于阈值
14     if (prev1 < prev2 && prev1 < curr_wave && prev1 < threshold)
15     {
16         if (curr_time > 5 && curr_time < 1000) // 合理区间，防止噪声
17         {
18             peak_times[pointer%5] = 6000 / curr_time; // 计时得到大约10ms触发一次中断
19         }
20     }
21 }
```

```

19                                     // curr_time*10得到间隔时间
20                                     // 60000/10=6000次每分钟
21                                     // %5实现指针的循环，不需要清零
22         pointer++;                    // 指针累加
23         curr_time = 0;                // 计到了峰值
24     }
25 }
26
27 // 更新历史值
28 prev2 = prev1;
29 prev1 = curr_wave;
30
31 // 继续计数
32 curr_time++;
33
34 int avg = (peak_times[0] + peak_times[1]
35           + peak_times[2] + peak_times[3]
36           + peak_times[4]) / 5;      // 用一个时间段内的均值
37 return avg;                          // 返回心率值
38 }

```

在计算 RR 间隔是用到了上面实验得到的估计时间差，方便计算起见近似为 10ms，得到了计算公式

为了更准确表达当前心率，需要减少过去心率的占比，可以使用指数加权平均法进行计算，扩大当前心率的权重；但是考虑到这个方法比较耗时，且整个中断例程已经非常庞大且耗时，我们没有使用这种方法而是用更简单快速的直接均值法

5. 结果展示

5.1 电极片位置

我们根据实验指导放置电极片进行信号采集，示意图如下：

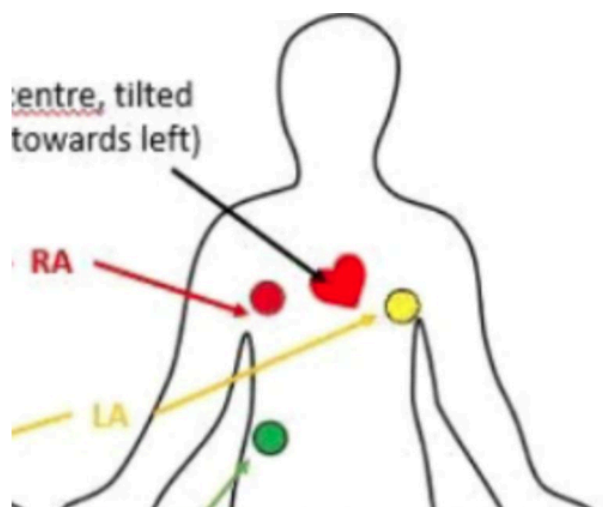


图 9 电极片位置

5.2 心电图展示

烧录程序到 MSP430，放置电极片，我们读取到心电信号如下图所示：

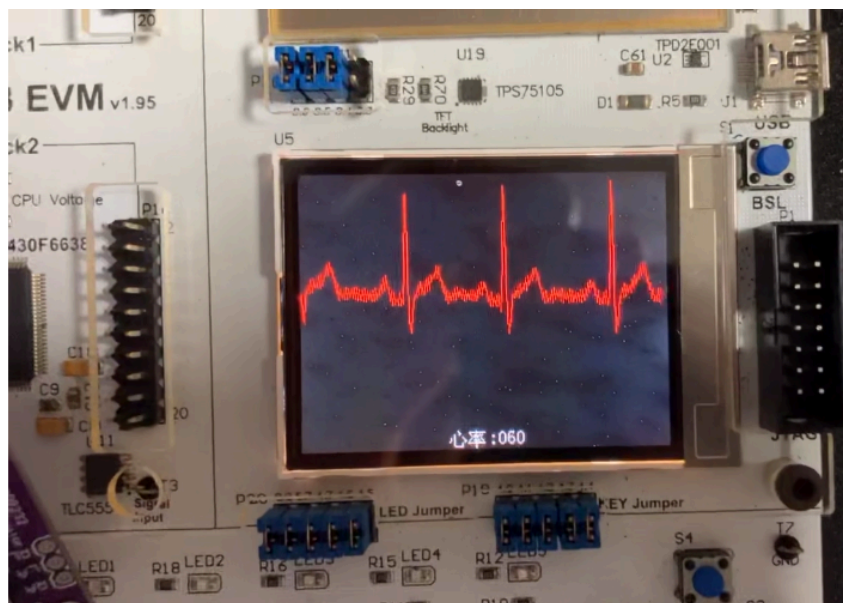


图 10 心电图结果展示

如上图所示，我们采集到人体的心率为 65 次/min。采集的过程中心率信号大概在 60-80 次/min 之间波动，根据我们查询的资料，正常成年人的心率水平大概在 60-100 次/min，误差在允许范围内。(具体的实验结果视频见附件。)

我们还利用健康手表与心电设备同时进行测量，对比二者结果的差异：

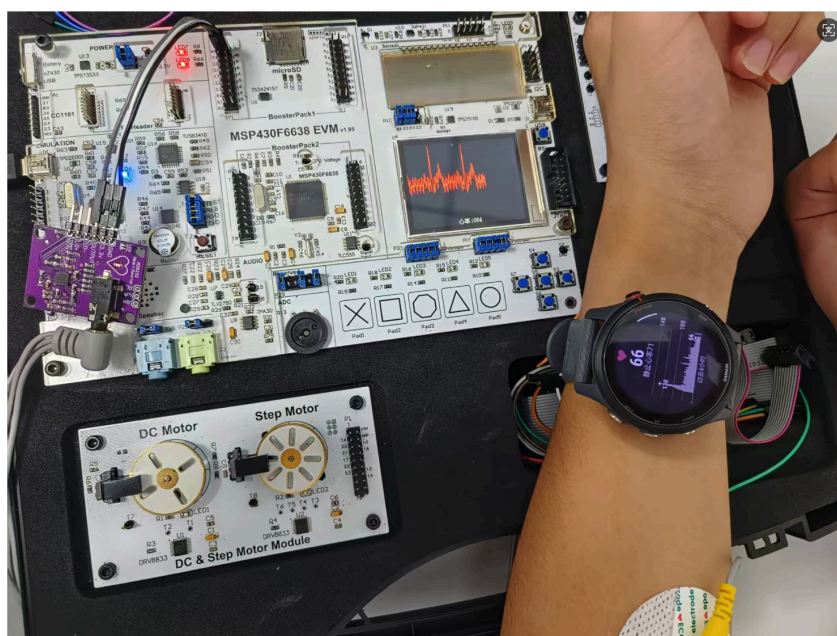


图 11 心电设备与电子健康手表结果对比

可以观察到实际的静息心率位 66 次/min，而我们的设备测量得到 64 次/min，误差较小，实验结果良好。

同时我们发现实际过程中由于桌面抖动也会导致心电图出现杂波，所以在实际测量时最好保证桌面平稳。

5.3 UART 串口通信结果及后处理

我们通过串口助手读取相应的心电数据，并将其转为 txt 文件用于后续的读取、滤波、绘制图像，将串口通信结果与绘制结果对比，进一步验证。

串口通信结果如下：

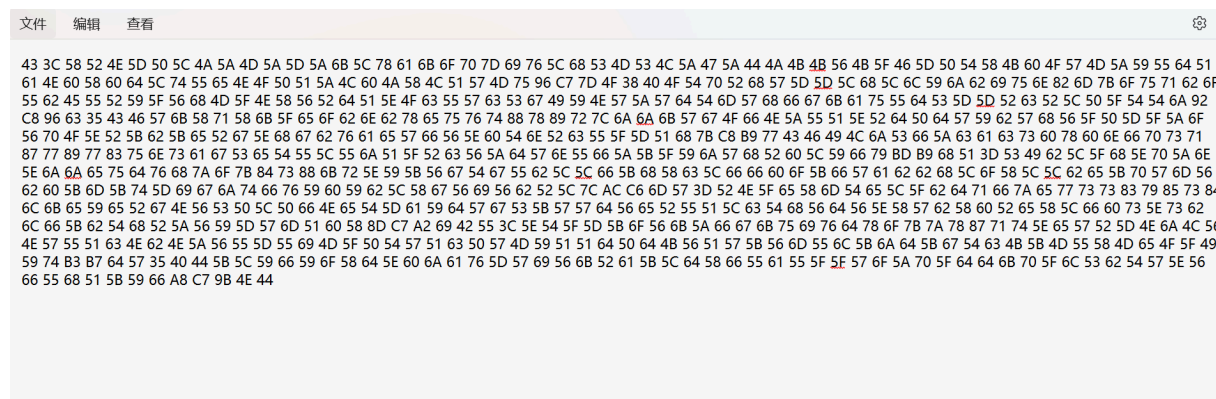


图 12 串口通信结果

python 代码如下：

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import pandas as pd
4
5 def read_hex_file_to_decimal(file_path):
6     with open(file_path, 'r') as file:
7         hex_data = file.read().strip() # 读取所有内容并去除首尾空白符
8
9     hex_list = hex_data.split() # 按空格分隔
10    decimal_list = [int(h, 16) for h in hex_list] # 转换为十进制
11    return decimal_list
12 # 定义滤波器，使用窗口均值滤波器
13 def moving_average_filter(data, window_size=5):
14
15     data = np.array(data)
16     filtered = np.convolve(data, np.ones(window_size)/window_size,
17                             mode='valid')
18     return filtered
19 # 读入文件
20 file_path = 'data.txt'
21 decimal_values = read_hex_file_to_decimal(file_path)
22
23 # 直接打印结果
24 # 创建时间数组（单位：秒），每个间隔10ms
25 time = np.arange(len(decimal_values)) * 0.01 # 转换为秒
26
27 # 绘制图表，使用时间数组作为x轴
28 plt.plot(time, decimal_values, linestyle='--', color='b')
```

```

28
29 # 设置x轴标签和标题
30 plt.xlabel('Time (s)')
31 plt.ylabel('ECG Values')
32 plt.title('Data with 10ms Interval')
33
34 # 显示图表
35 plt.show()
36 plt.figure(figsize=(10, 5))
37 plt.plot(decimal_values, linestyle='-', color='b')
38
39 #打印滤波后的结果
40 data_filtered = moving_average_filter(decimal_values, window_size=5)
41 time_filtered = np.arange(len(data_filtered)) * 0.01 # 转换为秒
42 # 绘制图表, 使用时间数组作为x轴
43 plt.plot(time_filtered, data_filtered, linestyle='-', color='b')
44
45 # 设置x轴标签和标题
46 plt.xlabel('Time (s)')
47 plt.ylabel('ECG Values')
48 plt.title('Data with 10ms Interval')
49
50 # 显示图表
51 plt.show()
52 plt.figure(figsize=(10, 5))
53 plt.plot(decimal_values, linestyle='-', color='b')

```

现在展示结果：

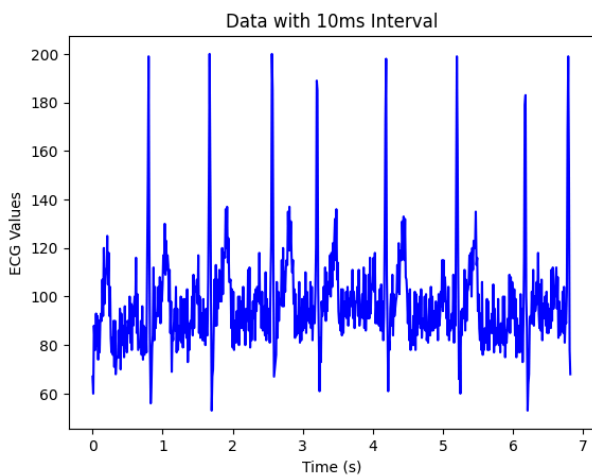


图 13 滤波前的 ECG

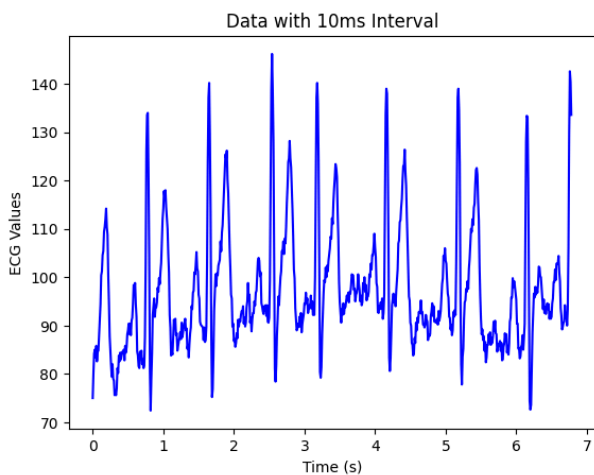


图 14 滤波后的 ECG

在上述过程中时间 $t = 6.83s$ (总共 683 个点, 间隔为 0.01s), 我们共检测到 8 次心跳信号, 得到

$$\text{心率} = \frac{8}{6.83} * 60 = 70 \text{次/min}$$

这与我们的测量结果基本吻合, 串口通信结果良好, 实验成功。

6. 困难与解决方案

6.1 问题一：代码可读性与小组合作

为了更好进行小组合作和方便代码的阅读，我们将代码分为：初始化、心率计算、心率展示、串口通信等多个模块化程序，这样既方便了我们的小组分工和合作，又提高了代码的可阅读性。

6.2 问题二：心率计算

我们最开始记录一次心跳信号的间隔，通过

$$f = \frac{60000(1\text{min对应}60000\text{ms})}{50} \times \text{count}(\text{count对应}50\text{ms内的心跳次数})$$

来计算心率，但是实际发现误差较大，我们发现我们在 intClock 函数中重新设置了时钟频率，这会导致当 Timer_A 比较值设为 50000 时对应的是 5ms 的计数时间。进一步，我们发现实际在 TimerA 中断中绘图也会占据一定时间消耗。我们重新对绘图进行计时，发现实际上调用一次中断所需的时间为 10ms。我们重新设计计算公式：

$$f = \frac{60000(1\text{min对应}60000\text{ms})}{10} \times \text{count}(\text{count对应}10\text{ms内的心跳次数})$$

6.3 问题三：心率平均计算

在实际过程中，我们为了能让心率较为准确且平稳的展示，我们在程序中计算 5 次心率，并进行平均减少误差。每次打印的心率值实际上是五次计算值的平均值，这样我们使得了心率波动更小，测量的结果更加准确，排除偶然因素干扰。

6.4 问题四：串口通信过程中的数据转化问题

在实际的串口通信过程中，由于我们一开始向 UCAxTxBUF 中写入的数据 value 为 12 位数据，但是实际上 UCAxTxBUF 为 8 位寄存器，这导致了每次传入的数据都发生了截断，影响了后续的绘图计算，我们使用(uint8_t)(value/20)强制转换 value 的值为 8 位以内，最后接收数据正确，绘制了实际的心电信号图像。